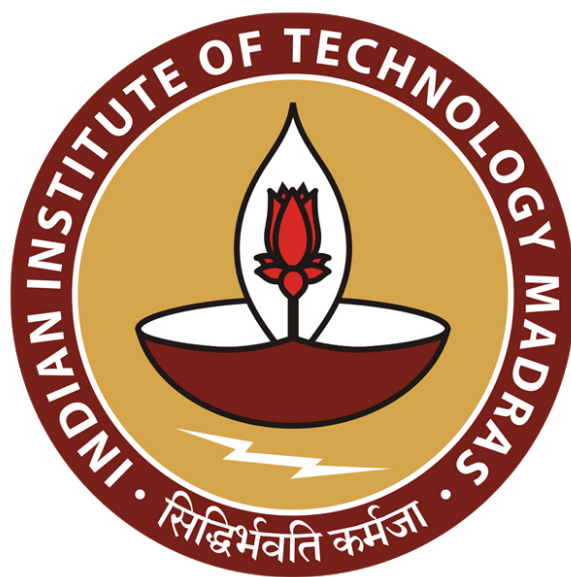




IIT Madras
ONLINE DEGREE

Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Advanced SQL

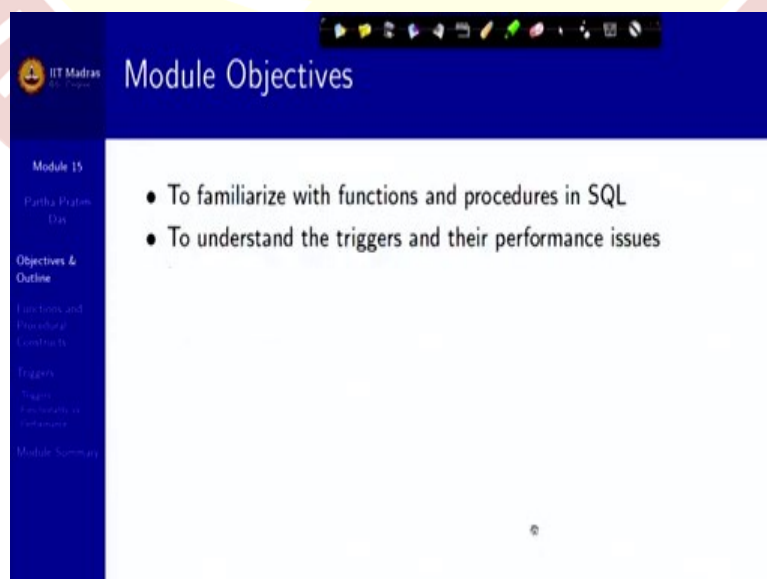
Welcome to module 15 of Database Management Systems in IIT Madras B.Sc. Online Program. We have so far been discussing about intermediate level features of SQL after the introductory features.

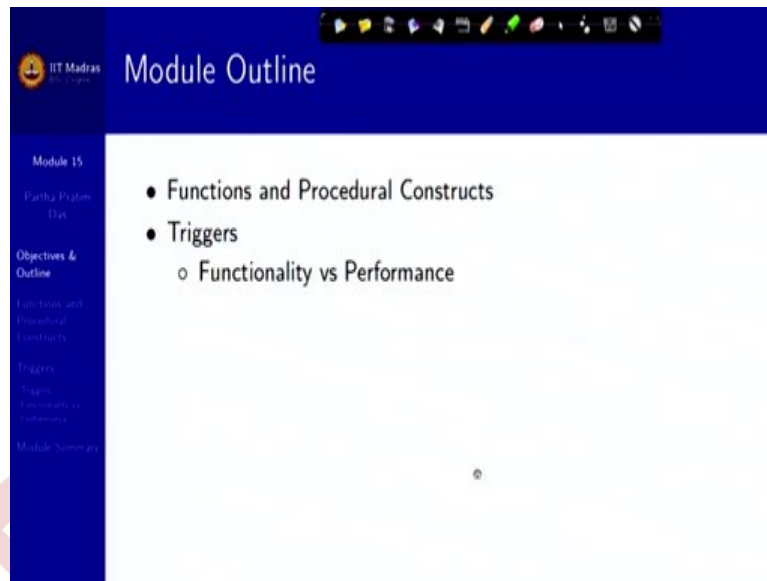
(Refer Slide Time: 0:34)



And in the last module we have talked about transactions, integrity constraints, additional data types and authorization.

(Refer Slide Time: 0:43)

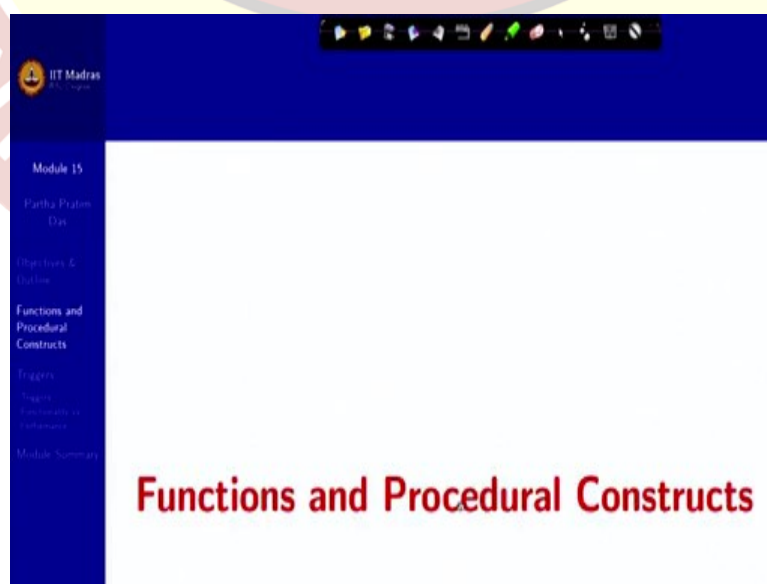


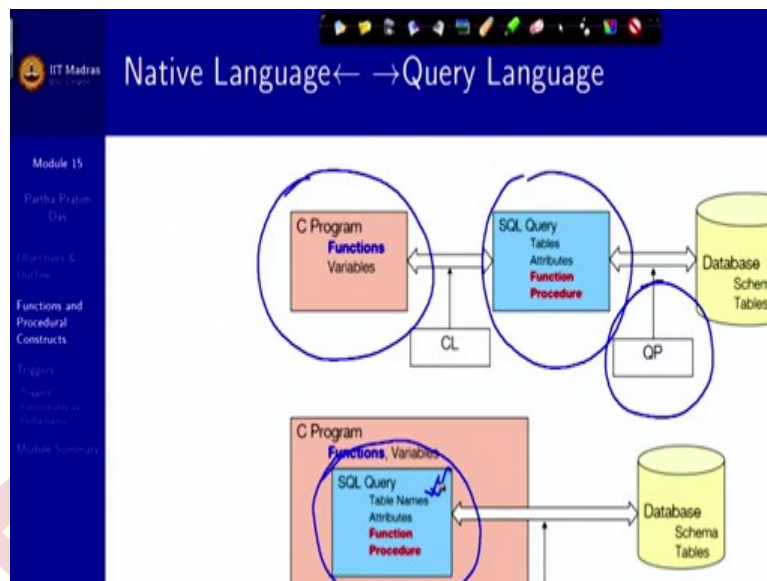


In this module I would like to take you to some of the other features which are you can say in some way for the initial design of or a lot of smaller applications this may not be critical but these are really powerful tools to extend the power of SQL to an anonymous level if I should say. So, I categorize them as the advanced features of SQL and these include functions and procedures in SQL.

Do not get surprised, I started by promising that SQL is a declarative language, it is but it has extensions which are procedural, so we will talk about that and another very important feature of triggers, we will see what do they mean and what kind of performance do they entail.

(Refer Slide Time: 1:34)





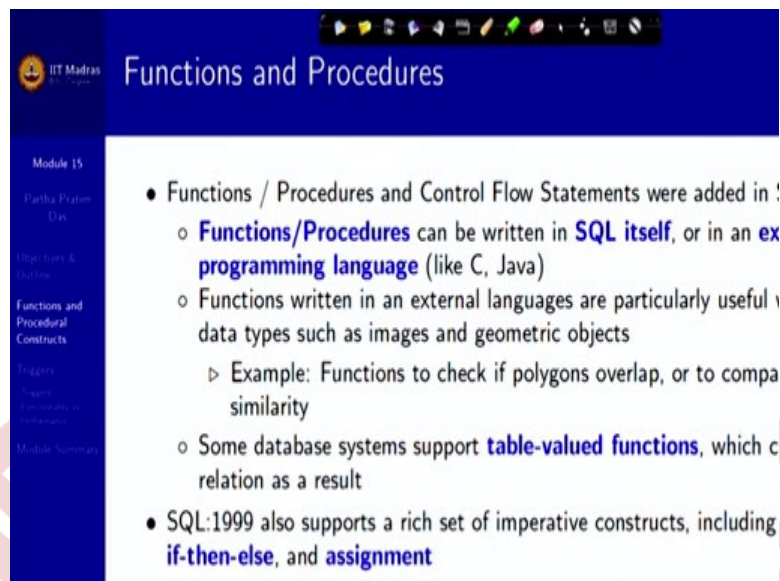
So, function and procedural constraints: So, one way that there are two basic ways these can play around; one is we talk about SQL as a query language and we talk about any other general purpose programming language like C or Java or Python as a native language. Somehow this term has come, I do not really understand why should it be called native, but it is.

So, what you have, if there are two basic models in which this is done; one is you have a conglomerate of C programs, functions and variables and so on. And you can actually invoke them from SQL, so you pass the data to the C function and the C function gives you the result back which is which is a kind of, this is, we often say is embedded or rather extensional programming using SQL. And SQL through the query processor continues to exclusively interact with the database.

The other model is instead of having them independently you have the SQL embedded as a part of the native language program code. So, it comes as a part of say you are writing a C function in between that there is a certain way to put SQL commands, SQL statements. So, that is more of a strongly embedded programming. Again, all the transactions with the database through the query processor will be done from the SQL.

But here they are not kind of independent entities, but kind of C program will control the invocation of the SQL operations and collection of results and so on, so there are two different types. And what actually SQL over period of time has done that within SQL itself it has given function and procedural features, so that even without using any native language you can do a lot of procedural programming in SQL.

(Refer Slide Time: 3:53)



Functions and Procedures

- Functions / Procedures and Control Flow Statements were added in SQL:1999
 - **Functions/Procedures** can be written in **SQL itself**, or in an **external programming language** (like C, Java)
 - Functions written in an external languages are particularly useful with data types such as images and geometric objects
 - ▷ Example: Functions to check if polygons overlap, or to compute similarity
 - Some database systems support **table-valued functions**, which return a relation as a result
- SQL:1999 also supports a rich set of imperative constructs, including **if-then-else**, and **assignment**

This started in SQL 1999 version, I just mentioned when I talked about the history. So, the function and procedures can be written in SQL itself, one option, or in an external programming language like the models we just saw, which could be C, Java, Python, anything. Functions written in external languages, so why, if we have a procedural extension then why do we still want external programming language interface.

The reason is the functions that are written in external language which we are earlier calling native language is particularly useful with spatialized data types, images, geometric objects, implementation of complex algorithms. You want to really make a complex design which is object oriented, you may have functions and procedures in SQL.

But is very difficult to do a object, pure object oriented design of certain paths which you can do in C++ or Java very easily, so we can do that and using this external programming technique you can take advantage of that design as a part of your database operations. So, these are some of the interesting things.

What has also been introduced is what is known as a table valued function, which means that a function in SQL can also return a table, normally functions return just simple values, but they can, in SQL the functions can be also return tables. And as we will see there are several constructs that are available.

(Refer Slide Time: 5:49)

SQL Functions

- Define a function that, given the name of a department, returns the number of instructors in that department:

```
create function dept_count (dept_name varchar(20))  
returns integer  
begin  
declare d_count integer;  
select count (*) into d_count  
from instructor  
where instructor.dept_name = dept_name  
return d_count;  
end
```
- The function `dept_count` can be used to find the department names of departments with more than 12 instructors:

So, I will just walk through these after explaining the function and procedure. Then for the remaining constructs which are very much like the imperative language constructs that is C or Java constructs, I will just walk through with the structure because detail, I mean, there are just little difference in the syntax, but their conceptualization, their logical model and control flow are very similar to the languages we know, details you can always pick up from the manual.

The first is to how to define functions. So, in SQL everything, any definition starts with create, so create function, the name of the function and the parameter, which you define, this is name and type. So, this is not a not an attribute, this is a function parameter now. The return type is written separately by returns, mind the “s” here. So, that dept count will take a varchar 20 value and give you an integer.

Then begin and end is the actual function body, this is a local variable, you can define several of them through declare, name and the type, and then you have a pure SQL query, so what is this query doing, select count star into d count, so it will count the number of records which satisfy the instructor dot dept name, that is the instructor dot dept name, the dept name field of the instructor table is equal to the dept name, which is a parameter given here.

So, we have given some name, passed some names, say physics, then it will check if the instructor table in the dept name column, whichever tuple has physics that will be counted. You are doing count star, so it gets one value and put that as d count and then you do a return on, you return d count, note the “s” is not there.

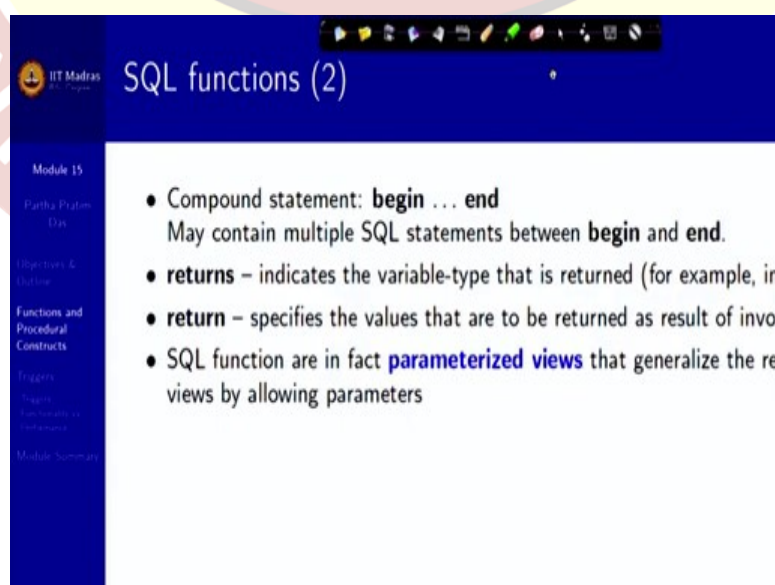
So, when you declare the function and you want to provide the return type, it is returns and when you actually return the value, it is just return, the “s” is not. So, this is what the function is. So, the way you can use it is, suppose, now you want to ah find out all departments which have more than, the budget for all departments which has more than 12 instructors, so department name, budget, that is coming from the department table, which is fine.

But how do you write the condition? You are using this dept count function, so you use dept count here and you are passing this department name, which is obtained from the department, dept name attribute of the department table as the parameter here. So, this function will count how many are there, give you that value through return d count and you will check whether it is greater than 12 or not; if it is greater than 12 it is true.

You take that department name and the corresponding budget in your output otherwise discard. So, that is how, very simply, so just the catch point is, even the declaration structure I find is very-very similar to what you do in the imperative language except for the syntax, but this gives you the basic way you can do data exchange between an, I mean, kind of a procedural variable to the select from where query structure of SQL.

And also how can you make use of that computed value in the select from where structure, so there is a data interchange between the declarative part of the language and the procedural part of the language. So, that is the basic thing to see otherwise rest of it is just matter of syntax, conceptually you will understand.

(Refer Slide Time: 10:13)



The screenshot shows a presentation slide from IIT Madras. The title is "SQL functions (2)". The slide content is as follows:

- Compound statement: **begin ... end**
May contain multiple SQL statements between **begin** and **end**.
- **returns** – indicates the variable-type that is returned (for example, in
- **return** – specifies the values that are to be returned as result of invol
- SQL function are in fact **parameterized views** that generalize the re views by allowing parameters

So these all we have discussed, so you can understand that basically function is a parameterized view, I mean, I could have created that view count star as to give me the value, but when I create the view I have to specify what is the department name; that cannot be a variable, but here that can be a variable, that can be a parameter, so this is a parameterized view and can be used for several other purposes as well.

(Refer Slide Time: 10:45)

Table Functions

- **Functions that return a relation as a result** added in SQL:2003
- Return all instructors in a given department:

```
create function instructor_of (dept_name char(20))  
returns table (  
    ID varchar(5),  
    name varchar(20),  
    dept_name varchar(20)  
    salary numeric(8,2) )  
returns table  
(select ID, name, dept_name, salary  
from instructor  
where instructor.dept_name = instructor_of.dept_name)
```

Table functions are functions that return a table which is very-very simple to see, one is in returns you provide the total table specification with, you do not need to give constraints and all that but just the names of attributes as well as their types and in returns now you have the actual query that gives you a table which matches the table you have promised you will return. So, that entire table will get return now.

So, the way to use it is since so instructor of, this is the table function instructor of, if you pass the department name music, will give you a table, so to make it clear to SQL that it is not a table by itself, but it is a table returned from a function, you say from table, this stable is used here to mean that a table function has to be invoked and that will give a table and that table will be used in the select query, so that is the kind of ah structure we are talking of here.

(Refer Slide Time: 12:09)

SQL Procedures

- The dept_count function could instead be written as procedure:

```
create procedure dept_count_proc (  
    in dept_name varchar (20), out d_count integer  
begin  
    select count(*) into d_count  
    from instructor  
    where instructor.dept_name = dept_name  
end
```
- Procedures can be invoked either from an SQL procedure or from em using the call statement.

```
declare d_count integer;  
call dept_count_proc('Physics', d_count);
```
- Procedures and functions can be invoked also from dynamic SQL

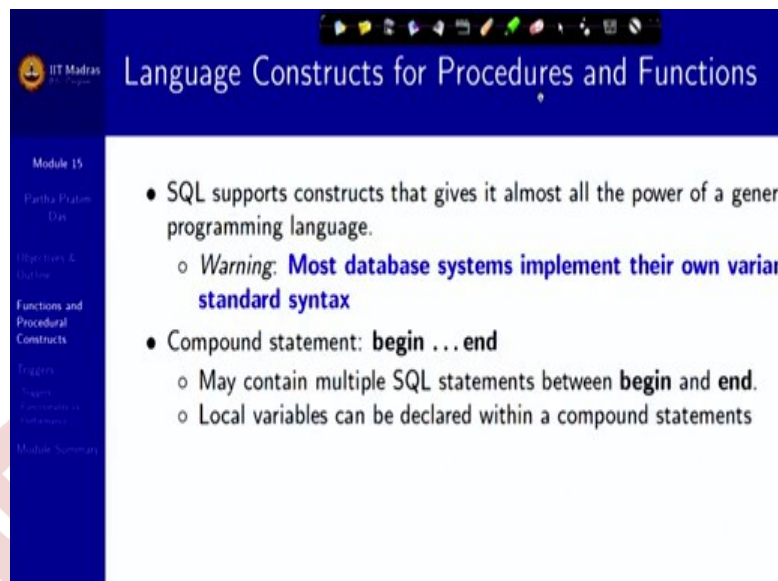
Now naturally as you have functions you also have procedures, which do not return anything, this does not have a return value, but it has any number of parameters and the parameters could be in and out, you can have as many as in parameters as many as out parameters and so on, but those are just parameters, there is no returns and we are doing the same kind of thing here and this is how the department name is coming in.

And what we have done here is you can see that this name, dept name is qualified by the name of the procedure, which says this is the procedural variable, because it is a parameter directly coming in and what you have gotten here is d count is a out parameter. Now, there is no separate return statement either because there is nothing, no specific value to return, so you will, you are setting the output parameter as a part of your SQL query.

And so when it reaches n it automatically, the control automatically returns to the caller and those out parameter value will be available. So, you can declare a integer variable and then you can use this procedure, the choice here is you have to use the word call, call tells that this is a procedure, because as a function it can occur as a part of an expression. So, as we said from table, it was implicitly understandable that is a function.

But here, when it is a procedure is not a part of a, it cannot be a part of an expression because it does not have a value, so you do a call, pass the parameters; so this is a in parameter, physics goes here, the count is computed and it comes out as d count. So, this is how, so you have functions as well as procedures, interestingly SQL procedures, functions and procedures can be overloaded like in C++ or Java. That is again you can use the same functional procedure name as long as the parameters differ in number or in the type.

(Refer Slide Time: 14:54)



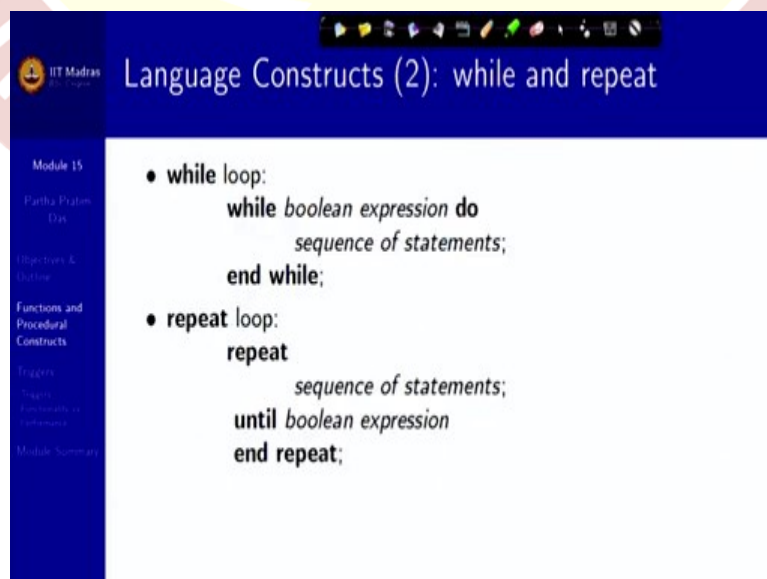
Language Constructs for Procedures and Functions

- SQL supports constructs that give it almost all the power of a general programming language.
 - **Warning:** Most database systems implement their own variant standard syntax
- Compound statement: **begin ... end**
 - May contain multiple SQL statements between **begin** and **end**.
 - Local variables can be declared within a compound statement

Now there are several typical procedural language constructs, I will just run through their what is available, but make sure that when you want to use it you have to look up your manual because this area function and procedures support of SQL is an area where many database systems are not following the standard, they are deviant, they support similar kind of constructs, but the name may be different syntax, may be different and certainly certain parts of the semantics will be different.

So, well, the first thing you have is naturally compound statement, which you have already seen **begin** and **end**, anything you put between **begin** and **end** is treated like a single statement as it is done in case of say pascal or in terms of curly braces in C, C++.

(Refer Slide Time: 15:56)



Language Constructs (2): while and repeat

- **while** loop:
`while boolean expression do`
`sequence of statements;`
`end while;`
- **repeat** loop:
`repeat`
`sequence of statements;`
`until boolean expression`
`end repeat;`

You have a while loop which is while Boolean expression do and you have a sequence of statements, so these are all SQL statements that you can put and then end is marked with end while. Repeat similarly has and so, so in while certainly you continue as long as the Boolean expression is true, so it means that it may or may not execute even once.

Repeat will necessarily execute for the first time, do a sequence of SQL statements until a Boolean expression is true, it will keep on repeating and the physical end, I mean, the little syntax difference with what we see in the common programming languages repeat until is very tense, but here you need a separate end repeat, to say that the scope of repeat has ended.

(Refer Slide Time: 16:55)

Language Constructs (3): for

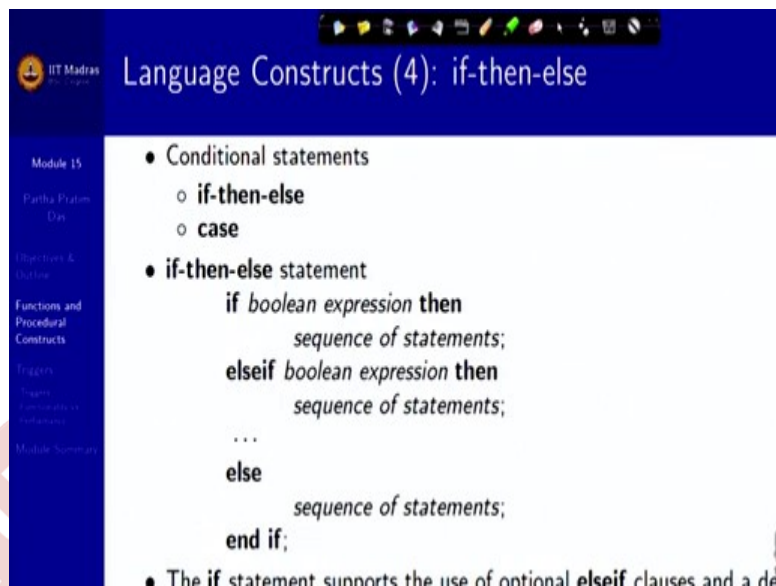
- for loop
 - Permits iteration over all results of a query
- Find the budget of all departments:

```
declare n integer default 0;  
for r as  
  select budget from department  
do  
  set n = n + r.budget  
end for;
```

You have a for loop, so for r as, so this is more like what you have in language like Python, not exactly in C, in C you have a very complex structure, but here you say for r as, select budget from department, so it is an SQL statement, so you have got the budget. So, what you have got r as is a relation, so you have got that relation.

So, now you can do a procedural statement which is an assignment, which is marked by set. So, you have declared a local variable n defaulted with 0, so you are incrementing n by this budget. So, this will keep on going as long as there are records in this budget relation that has come as r, so all those budgets will be get added.

(Refer Slide Time: 17:56)



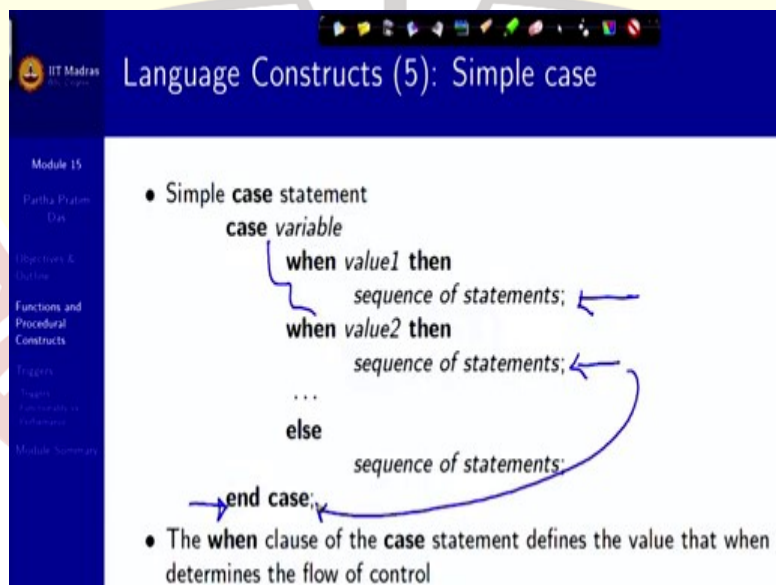
The slide is titled "Language Constructs (4): if-then-else". It lists "Conditional statements" with sub-points "if-then-else" and "case". It then shows the syntax for an "if-then-else statement":

```
if boolean expression then
    sequence of statements;
elseif boolean expression then
    sequence of statements;
...
else
    sequence of statements;
end if;
```

A bullet point at the bottom states: "The if statement supports the use of optional elseif clauses and a default else clause."

You have if-then-else if Boolean, I mean, these are if Boolean expression, then else-if and so on else is optional, else is default, you can very easily see and if you want some examples, you can look at the book, but I did not include because procedural part I am sure you all are very conversant with the particular different semantics of the procedural constructs.

(Refer Slide Time: 18:21)



The slide is titled "Language Constructs (5): Simple case". It lists a "Simple case statement" and shows its syntax:

```
case variable
    when value1 then
        sequence of statements;
    when value2 then
        sequence of statements;
    ...
    else
        sequence of statements;
end case;
```

Handwritten blue arrows point from the "sequence of statements;" lines to the "end case;" line, indicating the flow of control. A bullet point at the bottom states: "The when clause of the case statement defines the value that when determines the flow of control".

There are two types of case statement one which is very similar to switch in C, which is a case variable and there you write switch, then you write case, here you write case and then you write when, so when a value, so when the variable takes value 1 you do the first sequence of statements here, first sequence of statements here and then you come to the end case.

If the value is 2 you do, you jump over this, you do not do this, you do this and then come to the end, so that is exactly like the C switch. But there is a variant of this where, which is interesting is which is called searched case, which is you do a case and in when you do not write a value, you write an expression equal to a value.

So, this SQL expression can be almost anything, so that expression will get evaluated and the value of that, if it matches value one, then the first case condition sequence of case statements will be done and so on. So, this naturally, I mean, whatever you can write in the simple case statement you can write in the search case statement, but not the other way down, so this is a very powerful one somewhat different from what we know in the procedural languages.

(Refer Slide Time: 19:44)

The slide is titled "Language Constructs (7): Exception" and is from IIT Madras. It contains the following content:

- Signaling of exception conditions, and declaring handlers for exception

```
declare out_of_classroom_seats condition
declare exit handler for out_of_classroom_seats
begin
...
signal out_of_classroom_seats
...
end
```

A diagram shows a vertical line representing the execution flow. A bracket on the left side of the `begin` to `end` block indicates the scope of the handler. An arrow points from the `signal out_of_classroom_seats` statement to the right, indicating the point where the exception is raised.

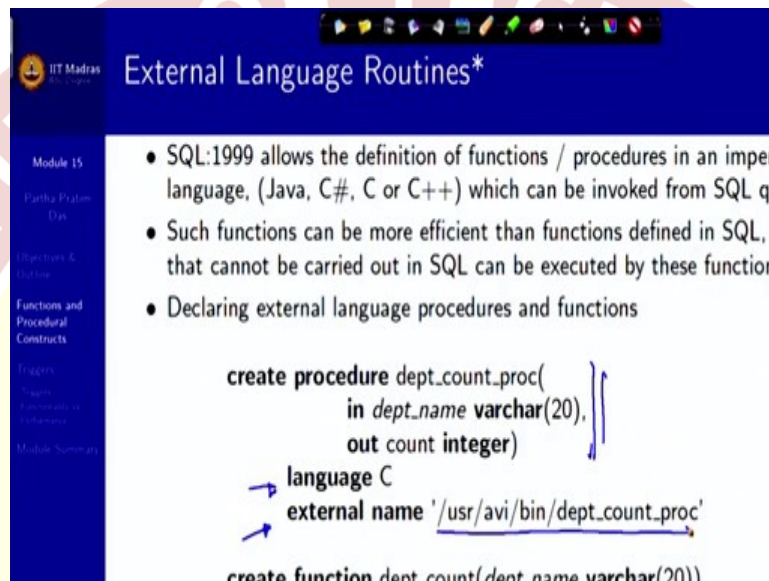
- The handler here is `exit` – causes enclosing `begin ... end` to be terminated
- Other actions possible on exception

It does support exception, so you can raise an exception by signal and when you raise the exception, the exception must be defined with the handler that is when you raise the exception it has to know where does the control go, what should be done, so you can define your own or there are some default handlers available, for example, what we are showing here is a declaration of an exception out of classroom seat.

So, I mean, you are allotting students to the classroom and you have exhausted all seats, so you cannot allot more students. So, the condition is the exit handler for this. So, exit handler is a handler which is already available in SQL, which does nothing but to terminate and exit this begin end whatever was action was being done here and that is a handler for this exception. So, this is how you declare an exception and then you associate a handler with it.

And whenever, so there is different logic that you would be doing here, here, so if you come across the situation where the exception condition has arose, for example, the class is full and still you are trying to add a student, then you will signal this exception, which will take you out of this, will terminate this and will take you to the handler. Handler being exit, it will not do anything further, but there could be other possible exceptions and handlers, I did not want to go into details of this they are just additional features.

(Refer Slide Time: 21:40)



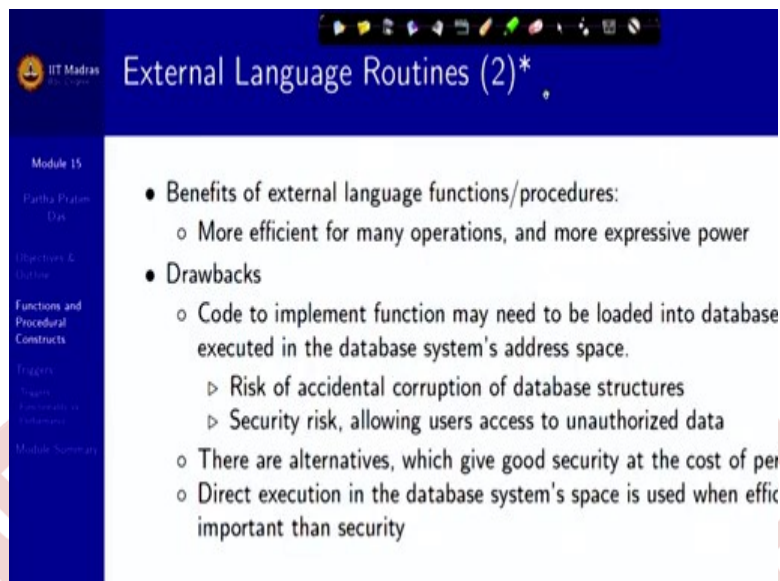
External Language Routines*

- SQL:1999 allows the definition of functions / procedures in an imperative language, (Java, C#, C or C++) which can be invoked from SQL queries
- Such functions can be more efficient than functions defined in SQL, and those that cannot be carried out in SQL can be executed by these functions
- Declaring external language procedures and functions

```
create procedure dept_count_proc(  
    in dept_name varchar(20),  
    out count integer)  
language C  
external name '/usr/avi/bin/dept_count_proc'  
  
create function dept_count(dept_name varchar(20))
```

Finally you can have external language routines as we saw and they are very relatively simple to deal with, this is again a create procedure, this part is nothing new, the body is different that you are saying this language C and the external name, so since it is a C program you have to say where does it exist, so you have to give a path name, this is some arbitrary path name, where that function can be found. So, the procedure and functions from external language also can be used in this manner.

(Refer Slide Time: 22:20)

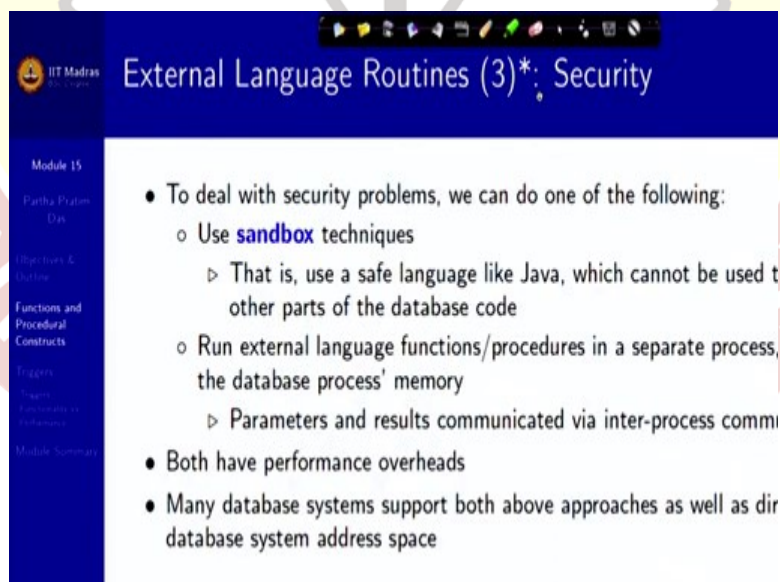


External Language Routines (2)*

- Benefits of external language functions/procedures:
 - More efficient for many operations, and more expressive power
- Drawbacks
 - Code to implement function may need to be loaded into database executed in the database system's address space.
 - ▷ Risk of accidental corruption of database structures
 - ▷ Security risk, allowing users access to unauthorized data
 - There are alternatives, which give good security at the cost of performance
 - Direct execution in the database system's space is used when efficiency is more important than security

There are, I talked about the benefits certainly is many operations become more expressive, more semantically clear as well as may be more efficient as well, but there are certainly risks because you are letting the control out of your hand, so it is going into a different space outside of the database system, so there are possibilities of accidental corruption or security risk and so on.

(Refer Slide Time: 22:52)



External Language Routines (3)*: Security

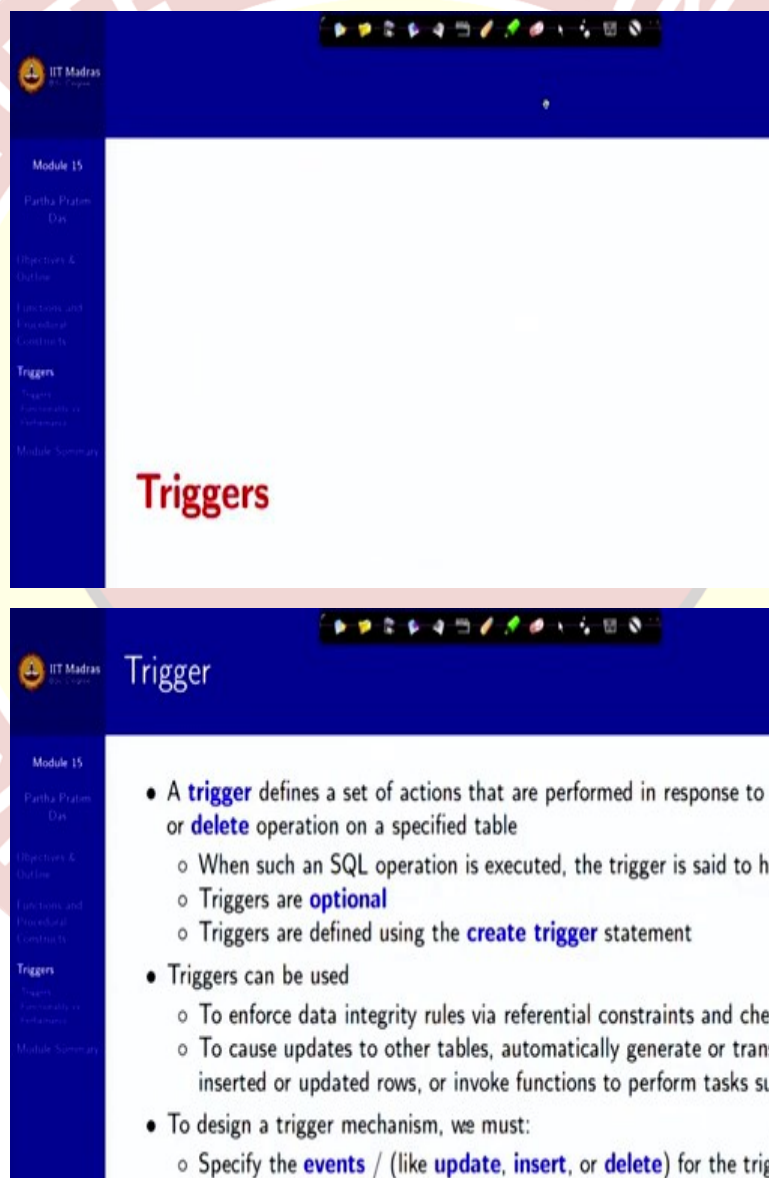
- To deal with security problems, we can do one of the following:
 - Use **sandbox** techniques
 - ▷ That is, use a safe language like Java, which cannot be used to access other parts of the database code
 - Run external language functions/procedures in a separate process, outside the database process' memory
 - ▷ Parameters and results communicated via inter-process communication
- Both have performance overheads
- Many database systems support both above approaches as well as direct execution in database system address space

So the care and different security choices like sandboxing will may have to be used, so you try to use languages, which are more safe, where it is not easy to break like, so I am not sure whether you are familiar with the safety of language is what it is called, like C is considered

extremely unsafe, several, particularly the string functions in C are very easy to break by buffer crashing and you can get access to the other data.

So, C now has a set of safe versions of the string functions. Java is relatively more safe because it does not give you direct access to the data, it always goes through the JVM and so on, so there are different choices you can make, but the bottom line is if you are using external language routines you will get efficiency, but may have to pay for through the lesser level of security, so be careful and you will have to take care of those things.

(Refer Slide Time: 24:00)



The image shows two screenshots of a presentation slide from IIT Madras. The top screenshot shows a slide titled "Triggers" in red text. The bottom screenshot shows a slide titled "Trigger" with a bulleted list of information about triggers.

- A **trigger** defines a set of actions that are performed in response to a **insert** or **delete** operation on a specified table
 - When such an SQL operation is executed, the trigger is said to have fired
 - Triggers are **optional**
 - Triggers are defined using the **create trigger** statement
- Triggers can be used
 - To enforce data integrity rules via referential constraints and check constraints
 - To cause updates to other tables, automatically generate or transform data, or invoke functions to perform tasks such as auditing
- To design a trigger mechanism, we must:
 - Specify the **events** / (like **update**, **insert**, or **delete**) for the trigger

Now, let us move on to something very different, which is also critical for database design, these are called triggers. Triggers are particularly the, I mean let me tell you the concern, the

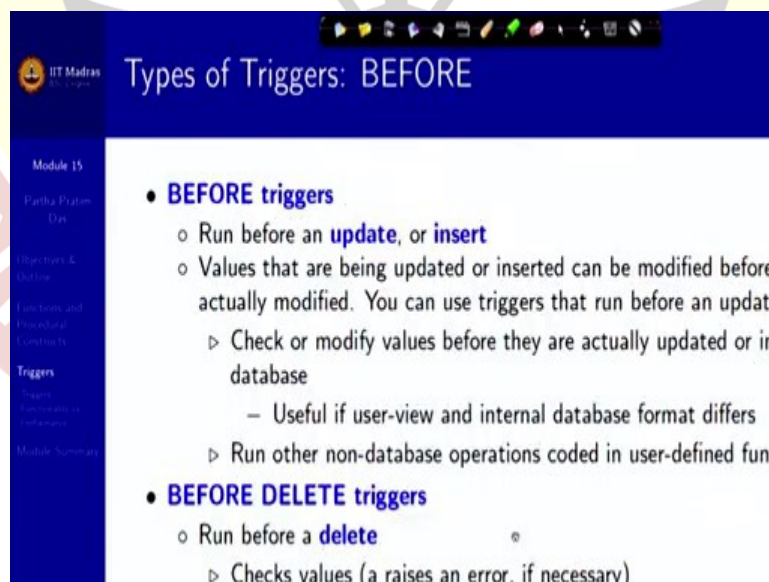
concern is a whole lot of things we are doing which are read the data, I mean, you can read the data that may be good, bad, whatever, but it has no consequence on the database.

But the moment you do modification to the data you write, you do insert or you do update or you do delete, then you have to make sure that several things had to remain, have to remain consistent, you have to, maybe if you delete something you need to record somewhere, you need to know who has done this delete for later on trace back, for auditing subsequently and so on.

So, these are insert, delete, update, these are called events, so when these events happen we want, we may want not necessarily every time, but when these events happen we may want that we would trigger an action that could be changing some other data, that could be flagging a message, anything.

So, triggers are often used to enforce data integrity via referential constraints, check constraints, to update other tables, like if you have to have this cascade you will need this kind of mechanism, so to specify trigger, we need to specify events, we need to specify when to fire the trigger is it before the execution of the event or after the execution of the event and what is action that has to be taken.

(Refer Slide Time: 25:56)



The screenshot shows a presentation slide from IIT Madras. The title is 'Types of Triggers: BEFORE'. The slide content is as follows:

- **BEFORE triggers**
 - Run before an **update**, or **insert**
 - Values that are being updated or inserted can be modified before actually modified. You can use triggers that run before an update
 - ▷ Check or modify values before they are actually updated or inserted into the database
 - Useful if user-view and internal database format differs
 - ▷ Run other non-database operations coded in user-defined functions
- **BEFORE DELETE triggers**
 - Run before a **delete**
 - ▷ Checks values (a raises an error, if necessary)

So, there are two types before trigger, so that runs before an update or insert happens, so for example, it may be that the user space uses certain format of the data and the database uses a different format of the data, so the user is trying to update the data given in the format in

which the user is using it, but before you can allow it to be updated you need to convert it to the format in which the database is actually keeping that data.

So, before trigger is a very good way of doing that you get to know that well update is going to happen, take that value, transform it, and then allow the update to happen. So, there could be several other reasons also. And there is a before delete triggers, this is specifically kept as a separate group because it runs before delete, because delete is the most dangerous, because you are removing something. So, you may have to check values, raise an error, see if this deletion is possible or not.

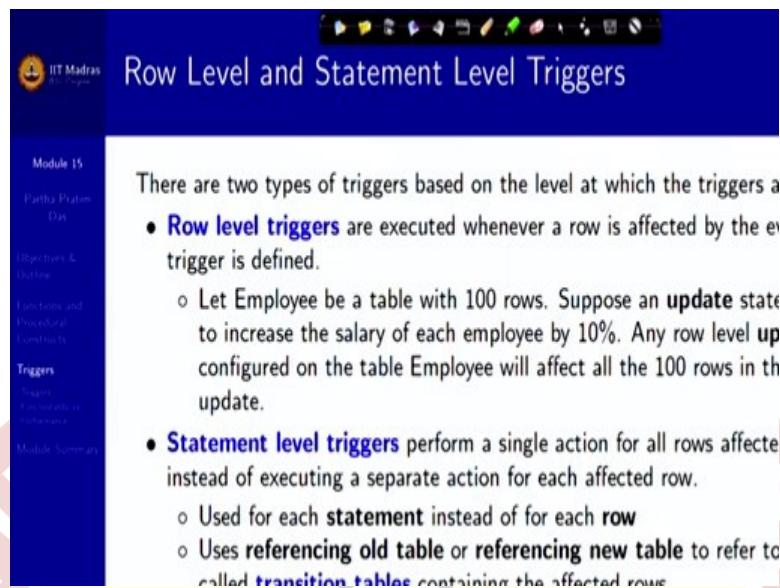
(Refer Slide Time: 27:02)

The screenshot shows a presentation slide with a blue header and a white content area. The header contains the IIT Madras logo and the title 'Types of Triggers (2): AFTER'. The content area lists the following points:

- **AFTER triggers**
 - Run after an **update, insert, or delete**
 - You can use triggers that run after an update or insert to:
 - ▷ Update data in other tables
 - Useful for maintain relationships between data or keep au
 - ▷ Check against other data in the table or in other tables
 - Useful to ensure data integrity when referential integrity c
 - appropriate, or
 - when table check constraints limit checking to the curren
 - ▷ Run non-database operations coded in user-defined functions
 - Useful when issuing alerts or to update information outsid

The second, so these are, timing wise this happens before the event, the others can happen after the event, which is run, which will be run after update, insert or delete, which can update other tables for referential integrity, which can check at data, which can convert some data into, for their data integrative purposes, you can do non database operations and so on. There are other types of triggers also, but not so widely used, I am not talking about them.

(Refer Slide Time: 27:37)



Row Level and Statement Level Triggers

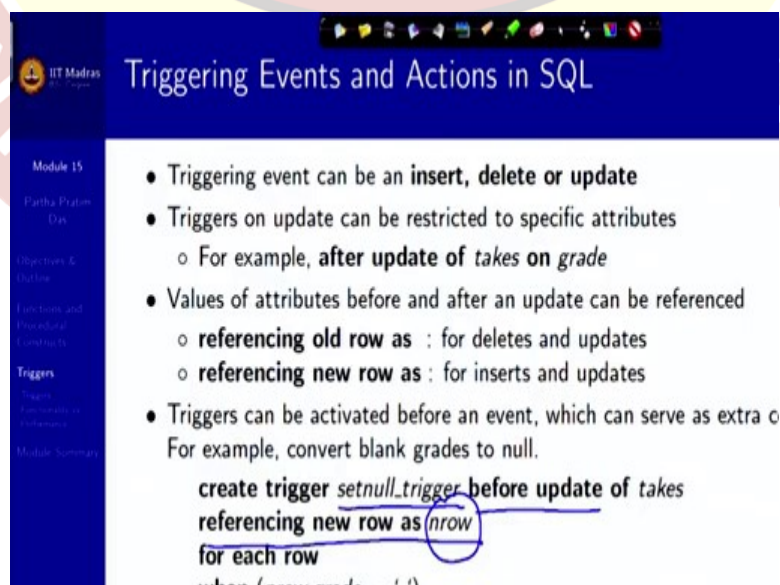
There are two types of triggers based on the level at which the triggers are defined.

- **Row level triggers** are executed whenever a row is affected by the event for which the trigger is defined.
 - Let Employee be a table with 100 rows. Suppose an **update** statement is used to increase the salary of each employee by 10%. Any row level trigger configured on the table Employee will affect all the 100 rows in the table.
- **Statement level triggers** perform a single action for all rows affected by a statement instead of executing a separate action for each affected row.
 - Used for each **statement** instead of for each **row**
 - Uses **referencing old table** or **referencing new table** to refer to the table before or after the event. These tables are called **transition tables** containing the affected rows.

Now, triggers can be associated with either rows or with statements, so if it is associated with a statement, SQL statement, then it applies to all rows of a particular statement result. If it is said on the row, then it applies to a specific row, so if you do a statement level with a row level, if you execute a statement with the row level trigger which has 100 rows, then those trigger will happen hundred times.

So, you have to be careful in terms of doing that. And in writing the action you can actually refer to the table before change and table after change by referencing old table or referencing new table.

(Refer Slide Time: 28:26)



Triggering Events and Actions in SQL

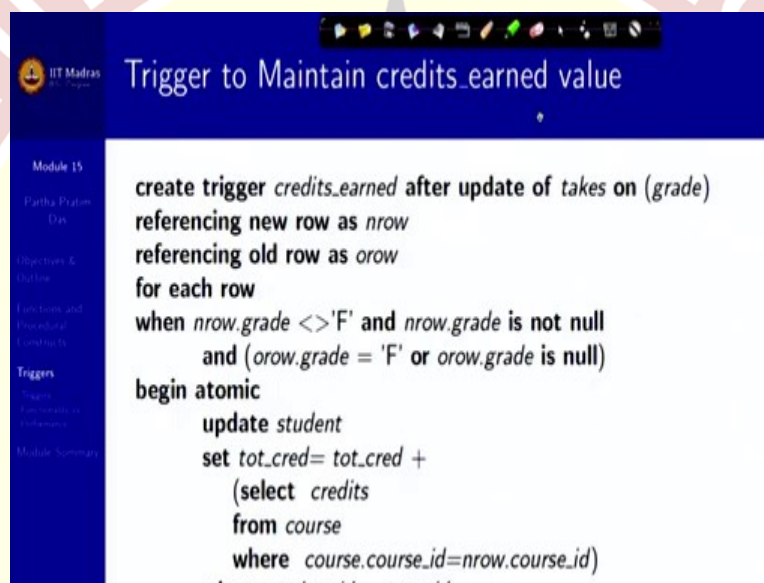
- Triggering event can be an **insert, delete or update**
- Triggers on update can be restricted to specific attributes
 - For example, **after update of takes on grade**
- Values of attributes before and after an update can be referenced
 - **referencing old row as** : for deletes and updates
 - **referencing new row as** : for inserts and updates
- Triggers can be activated before an event, which can serve as extra condition. For example, convert blank grades to null.

```
create trigger setnull_trigger before update of takes
referencing new row as (nrow)
for each row
when (nrow.grade = '')
```

So, here this is some example of a trigger, create trigger, everything in SQL starts with create, create trigger before update, so the trigger name, you know the trigger timing, you know the trigger event, so three things are known. So, what you are saying is referencing new row as n row, so n row is the new row that is going to happen with the update.

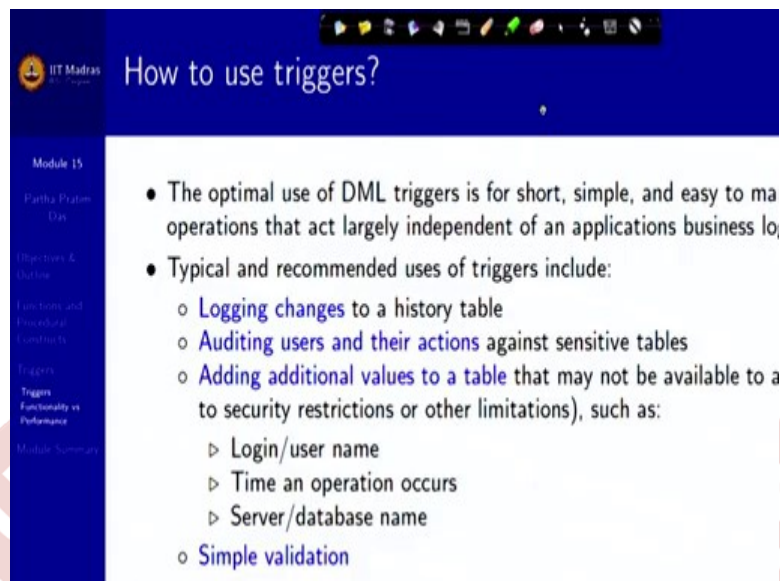
And for each row you are saying new row grade, that is the grade already available in the new row, if it is blank make it null, if it is blank make it null and then you do the update otherwise, maybe that update logic will not work properly. And you have this as atomic because it cannot, it has to happen or not happen, it cannot be halfway.

(Refer Slide Time: 29:40)



There is another little bit more detailed example, which I will leave for you to read while you work and figure out what it is doing. It is pretty simple straight forward, it is trying to compute the total credit of a student based on pass or failure.

(Refer Slide Time: 30:04)



How to use triggers?

- The optimal use of DML triggers is for short, simple, and easy to maintain operations that act largely independent of an application's business logic.
- Typical and recommended uses of triggers include:
 - Logging changes to a history table
 - Auditing users and their actions against sensitive tables
 - Adding additional values to a table that may not be available to a user (due to security restrictions or other limitations), such as:
 - ▷ Login/user name
 - ▷ Time an operation occurs
 - ▷ Server/database name
 - Simple validation

Now, the question is when to, how to use triggers because figure is kind of, is a point where you can do something. Now there are certain situations and there has been lot of cases where triggers have been wrongly used or used at wrong places causing severe damage, particularly in terms of performance, so some typical situations where triggers are good are logging changes in a history table, that is you want to write down that this has happened, this has happened, this has happened for things that are important to remember.

Then auditing users and their actions, I said you can write down who has done it, so then later on that person can be questioned for having done what he or she has done, adding additional values to the table, referential integrity, login, user name, time of operation and so on, simple validations, data sanity and so on. So, these are some of the typical ways to use trigger.

(Refer Slide Time: 31:10)

How not to use triggers?

- Triggers are like Lays: *Once you pop, you can't stop*
- One of the greatest challenges for architects and developers is to ensure
 - triggers are used only as needed, and
 - to not allow them to become a one-size-fits-all solution for any data that might happen to come along
- Adding triggers is often seen as faster and easier than adding code to the database, but the cost of doing so is compounded over time with each added line of code

Source: *SQL Server triggers: The good and the scary*

But you have to be careful to remember and that is severely important is when not to use trigger, it is like the Lays, once you pop you cannot stop, you open a packet of Lays, you cannot have one and stop, you keep on, so triggers are like that, once you start using them developers have shown the propensity to keep on using more and more of triggers.

(Refer Slide Time: 31:37)

How to use triggers? (2)

- Triggers can become dangerous when:
 - There are too many
 - Trigger code becomes complex
 - Triggers go cross-server - across databases over network
 - Triggers call triggers
 - Recursive triggers are set to ON. This database-level setting is set to ON by default
 - Functions, stored procedures, or views are in triggers
 - Iteration occurs

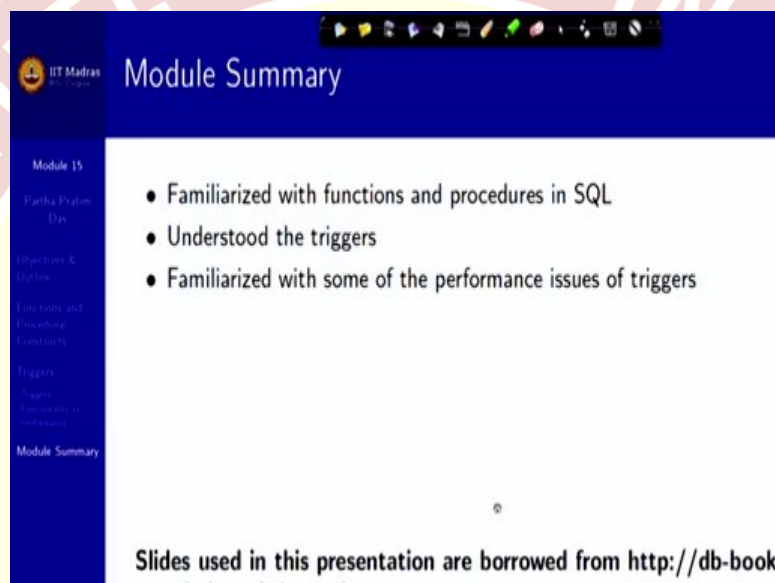
Source: *SQL Server triggers: The good and the scary*

So, on red I have tried to put some of the things you must not be doing like that you should not have too many triggers, they should be used very judiciously. Trigger code should be simple, they may not, must not be complex. They should not refer to crossover, cross-server databases; go across databases and so on. Triggers in turn should not call triggers, chain effect, lot of them.

Recursive trigger normally is set off, so never set it on; that is trigger quality itself. Functions, stored procedure, views these are expensive, do not have them in triggers. Iteration, avoid iteration in triggers and so on. So, the whole idea is make triggers smart, selective, short but very effective powerful weapon to maintain the consistency of your database.

We plan to have a tutorial on triggers to give you some more details, walk through examples and so on, some case studies the team has prepared and one of the tutorials will talk more about how to and how not to use triggers.

(Refer Slide Time: 32:57)



So, that brings us to the end of our module, so we have seen the functions and procedures in SQL and we have taken note of the triggers. Thank you very much for your attention and see you in the next module.